

Python Programming

A practical study reference with foundational concepts, examples, and common review questions.

1. Foundations

Python is a high-level, dynamically typed language. Names refer to objects; assignment binds a name rather than copying an object. Indentation defines code blocks. The common CPython implementation compiles source to bytecode and executes it on a virtual machine.

Built-in types include int, float, bool, str, list, tuple, dict, set, and None. Lists and dictionaries are mutable; strings and tuples are immutable. A shallow copy duplicates the outer collection but can still share nested objects.

2. Control Flow and Functions

Use if/elif/else for branching, for to iterate over iterables, and while for condition-based repetition. break exits a loop; continue advances it. Functions use def and return values. Arguments may be positional or keyword-based, and type hints document intended types without default runtime enforcement.

Default arguments are evaluated once when a function is defined, so avoid mutable defaults. Closures retain access to enclosing variables. Generators use yield to produce values lazily and can process large inputs without materializing all results.

```
def add_tag(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags
```

3. Collections and OOP

A list is an ordered mutable sequence; a tuple is ordered and immutable; a set contains unique hashable values; a dict maps unique hashable keys to values. Use enumerate for indexes, zip for parallel iteration, and comprehensions for clear transformations.

A class defines state and behavior; an instance is an object made from it. __init__ initializes instances. Inheritance models an is-a relationship; composition combines collaborating objects. Prefer small cohesive classes and explicit ownership of mutable state.

4. Files, Errors, Testing

Use with to manage files and other resources; specify text encodings. pathlib handles paths, while json and csv handle common data formats. Catch only exceptions you can address, and preserve useful error context.

Write focused tests for normal behavior, edge cases, and failures. unittest is built in; pytest is widely used. Avoid eval on untrusted data, protect credentials, validate external input, and keep dependencies maintained.

5. Common Questions

List versus tuple: lists can be changed; tuples cannot. is checks object identity, while == compares values. Use is None for a None check. A generator is an iterator that yields values on demand. A virtual environment isolates project dependencies.